1　第 1 讲：优化的边界（计算复杂性）

题 1（概念题 ⋆）　请简述 P 类问题和 NP 类问题的定义，并解释"P ⊆ NP" 为什么成立。

题 2（概念题 ⋆）　解释以下概念之间的区别：

(a) 可计算问题与不可计算问题

(b) P 类问题与 NP 类问题

(c) NP-Hard 问题与 NP-Complete 问题

题 3（概念题 ⋆⋆）　用停机问题（Halting Problem）的不可判定性来解释" 优化的上限"。请给出停机问题不可判定的简要证明思路（对角化论证）。

题 4（计算题 ⋆）　考虑旅行商问题（TSP），n 个城市。

(a) 暴力枚举算法的时间复杂度是多少?

(b) 当 $n = 15$ 时，暴力枚举大约需要计算多少条不同的路径?

(c) 使用动态规划（Held-Karp 算法）可以将时间复杂度降到多少?

题 5（概念题 ⋆⋆）　什么是多项式时间归约（polynomial-time reduction）? 请说明如何将 3-SAT 问题归约到独立集问题（Independent Set），并解释这如何证明独立集问题是 NP-Complete 的。

题 6（思考题 ⋆⋆）　课程提出" 上限-下限" 框架：上限是不可计算，下限是暴力枚举 $O(2^n)$。请解释：

(a) 为什么说" 科研的本质是发现结构、发展针对性方法"?

(b) 请用背包问题为例，说明如何通过发现不同层次的结构（如贪心近似、DP 精确解、FPTAS）在上限和下限之间找到好的算法。

题 7（思考题 ⋆⋆⋆）　GPU 并行可以改变我们对复杂度的看法。假设有一个 $O(n^2)$ 的算法 A 可以完全并行化（每步独立），和一个 $O(n \log n)$ 的算法 B 本质上是顺序的。

(a) 在拥有 p 个并行处理器的情况下，算法 A 的并行时间复杂度是多少?

(b) 在什么条件下（n 和 p 的关系），并行化的 A 比顺序的 B 更快?

(c) 这对" 最优算法" 的概念有何启示?

题 8（计算题 ⋆⋆ 归约手算：图着色 →3-SAT）　考虑图 G 有 3 个节点 $\{v_1, v_2, v_3\}$ 和 3 条边 $\{(v_1, v_2), (v_2, v_3), (v_1, v_3)\}$（完全图 K_3）。使用 3 种颜色 $\{R, G, B\}$ 的图着色问题。

将 3-着色问题编码为 SAT：为每个节点 v_i 和颜色 c 引入布尔变量 $x_{i,c}$（表示"v_i 被染为颜色 c"）。

(a) **至少一种颜色**：为每个节点写一个子句，保证它至少被染一种颜色。例如 v_1 对应 $(x_{1,R} \vee x_{1,G} \vee x_{1,B})$。写出全部 3 个子句。

(b) **至多一种颜色**：对每个节点，任意两种颜色不能同时为真。例如 v_1 不能同时为 R 和 G：$(\neg x_{1,R} \vee \neg x_{1,G})$。写出全部 $3 \times \binom{3}{2} = 9$ 个子句。

(c) **相邻不同色**：对每条边和每种颜色，两个端点不能同色。例如边 (v_1, v_2) 颜色 R：$(\neg x_{1,R} \vee \neg x_{2,R})$。写出全部 $3 \times 3 = 9$ 个子句。

(d) 共多少个变量？多少个子句？手动求解该 SAT 实例（找到所有满足赋值），验证它们恰好对应 K_3 的所有合法 3-着色方案。K_3 有多少种合法着色？

(e) 如果改为 K_4（4 个节点的完全图，仍用 3 种颜色），变量数和子句数分别变为多少？该 SAT 实例是否可满足？为什么？

题 9（计算题 ⋆⋆ 归约手算：子集和 →SAT）　子集和问题：给定集合 $S = \{3, 5, 7\}$ 和目标 $T = 8$，是否存在子集的和恰好为 T？

用布尔变量 x_i（$x_i = 1$ 表示选第 i 个元素）。

(a) 写出约束：$3x_1 + 5x_2 + 7x_3 = 8$。枚举所有 $2^3 = 8$ 种赋值，哪些满足？

(b) 将整数约束转化为二进制表示的逻辑电路（加法器），再将电路转化为 CNF 子句（Tseitin 变换）。画出 $3x_1 + 5x_2$ 的二进制加法电路（$3 = 011_2$, $5 = 101_2$, 和最多 1000_2）。

(c) 对一个全加器（3 输入 1 位加法：$a + b + c_{\text{in}} = 2c_{\text{out}} + s$），写出 s 和 c_{out} 的逻辑表达式，并转化为 CNF（Tseitin 变换：引入中间变量，每个门 ≤ 3 个子句）。

(d) 这个归约说明了什么？（子集和是 NP-Complete，因此它可以被编码为 SAT。）

题 10（⋆）　SAT 求解器体验

用 Python 实现一个暴力 SAT 求解器：输入 CNF 公式（子句列表），枚举所有 2^n 种变量赋值，检查是否满足。

(a) 对以下 CNF 公式求解：$(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$。

(b) 随机生成含 n 个变量、m 个子句（每个子句 3 个文字）的 3-SAT 实例。测试 $n = 10, 15, 20$ 时的求解时间。

(c) 固定 $n = 20$，画出"可满足概率"随 m/n 比值变化的曲线（子句密度相变现象）。